

Manual de Arquitectura y Estructura del Proyecto

Agente IA (Orquestador)

26 de junio de 2026

Índice

1. Introducción	2
2. Directorios Principales	2
3. Archivos de Configuración y Scripts Raíz	2
4. Conclusión	3

1. Introducción

Este manual describe la estructura de directorios y archivos presentes en el espacio de trabajo de Electrónica. El entorno está diseñado bajo una arquitectura de tres capas, que separa la lógica probabilística (LLMs) de la ejecución determinista (scripts), y también contiene recursos para IoT, Diseño de Circuitos Integrados (EDA) y Redacción Académica.

2. Directorios Principales

- `.agent/`: Contiene las instrucciones del sistema para el agente de IA, definiendo su marco operativo, restricciones de entorno y protocolos de actuación.
- `directives/`: Capa 1 de la arquitectura. Contiene manuales de operación (SOPs) en formato YAML. Definen *qué* debe hacerse (ej. `scrape_website.yaml`, `git_update.yaml`).
- `execution/`: Capa 3 de la arquitectura. Contiene scripts deterministas en Python con una sola responsabilidad (ej. `env_diagnostic.py`, `alert_user.py`).
- `.tmp/`: Directorio temporal utilizado por la capa de orquestación para almacenar estados de ejecución (como `run_state.json`) y artefactos temporales de procesamiento.
- `docs/`: Documentación técnica y académica (ej. proyectos de circuitos, fuentes de alimentación, apuntes de EDA).
- `cursos/`: Material didáctico, notas de cursos de Electrónica, apuntes de Laboratorio y Tesis.
- `chroma_db/`: Base de datos vectorial autogenerada, utilizada por el sistema RAG (Retrieval-Augmented Generation) para la memoria y búsqueda semántica del agente.
- `Agente_EDA/`: Recursos específicos para el agente EDA, como esquemas (schemas) de validación y pruebas para la generación de netlists/BOM hacia EasyEDA.

3. Archivos de Configuración y Scripts Raíz

- `AGENTS.md`: Reglas globales del proyecto, convenciones de código y estructura general del espacio de trabajo.
- `.env`: Archivo para almacenar credenciales locales (ej. `GROQ_API_KEY`) y configuración de entorno sin hardcodear datos sensibles.
- `requirements.txt`: Dependencias del sistema necesarias para ejecutar los scripts deterministas (ej. `psutil`, `PyYAML`).
- `rag_system.py`: Script del Chatbot RAG. Vectoriza documentos (TEX, MD, PDF) en ChromaDB y gestiona las respuestas usando el LLM vía Groq.

- `agent_eda.py`: Agente especializado en EDA. Extrae netlists y BOM de archivos LaTeX (circuitikz) para exportar hacia EasyEDA Standard.
- `clean_latex.py`: Utilidad para eliminar archivos auxiliares y temporales de compilación de LaTeX (.aux, .log, .out, etc.).
- `md_to_pdf.py` y `merge_pdfs.py`: Utilidades para la conversión y manipulación de documentos PDF.
- `ren_archivos.py`: Script para el renombrado masivo de archivos eliminando cadenas específicas.
- `test_generator.py`: Scripts de prueba (sin dependencias externas) para el generador JSON de EasyEDA.
- `db_state.json`: Archivo que lleva el registro del estado y actualizaciones incrementales de la base de datos RAG vectorial.
- `opencode.json`: Archivo de configuración que enlaza las instrucciones del agente alojadas en `.agent/`.
- `git-update.sh` y `update_repo.sh`: Scripts de automatización en bash para sincronización y actualización del repositorio Git.

4. Conclusión

Esta estructura garantiza una alta modularidad. El agente IA actúa como "middleware" o capa de orquestación (Capa 2), leyendo intenciones, buscando las instrucciones en `directives/`, delegando el trabajo duro a `execution/` y documentando hallazgos en `chroma_db/` o en archivos `.tex`. Todo ello cumpliendo con las reglas de codificación y formato estipuladas.